inference-engine

An LLM serving engine built from scratch

Brief

1 The project in one page

`inference-engine` turns a trained language model into a service. It is an HTTP server that many clients can send prompts to simultaneously, that streams words back as they are generated, and that shares one machine's memory between all of them under a coherent policy.

It is Python and PyTorch, running GPT-2 family models (`gpt2`, `distilgpt2`) on CPU. Hugging Face supplies the trained weights and the tokenizer. Everything that makes it a serving *engine* is implemented in the repository: the forward pass, the attention, the memory allocator, the scheduler, the sampler, the speculative decoder, the HTTP surface.

Why a project like this exists

Serving a language model is not a machine learning problem. It is an operating systems problem wearing a machine learning hat: a memory allocator, a scheduler, cache coherence, admission control, preemption. The three mechanisms that define a modern serving engine (paged KV cache, continuous batching, speculative decoding) are all systems techniques. This project rebuilds them small enough to hold in one head, verifies them against a reference implementation, and measures them honestly on the hardware actually available.

Area	Lines	Contents
`engine/`	815	cache, block manager, scheduler, model, sampling, speculative
`server/`	206	FastAPI, SSE, CLI
`tests/`	731	7 modules, 55 tests
`benchmarks/`	271	harness plus committed JSON results

2 The three ideas, from first principles

A language model can do exactly one thing: given some tokens, predict the next one. Text is produced by looping: predict, append, predict again. Every word you see from a chatbot cost one full pass through the model.

2.1 Why a KV cache exists

Each model layer performs *attention*: every position looks back at every earlier position, matching a *query* against each one's *key* to decide how much of its *value* to take. Predicting token 501 needs the keys and values of tokens 1 through 500. Recomputing them each step makes generating n tokens cost $O(n^2)$.

The cache, and the problem it creates

A token's key and value depend only on the tokens before it. Attention is *causal*: the past cannot see the future, so the past never changes. Compute each position's keys and values once, keep them, and every later step reads them back. Generation becomes $O(n)$ and practical.

That store is the **KV cache**, and it converts repeated computation into retained memory. For `gpt2` at this project's defaults the cache is about 604 MB, against roughly 500 MB of model weights. The weights are paid for once; the cache is paid for per user, per token, forever. **The KV cache, not the model, is what runs out first**, and managing it is what a serving engine is.

2.2 Idea one: page the cache

The naive cache gives each sequence one contiguous region sized for the longest answer it might produce. This over-reserves for tokens that may never exist, lets one long request block short ones, and leaves freed gaps that are the wrong shape for the next arrival.

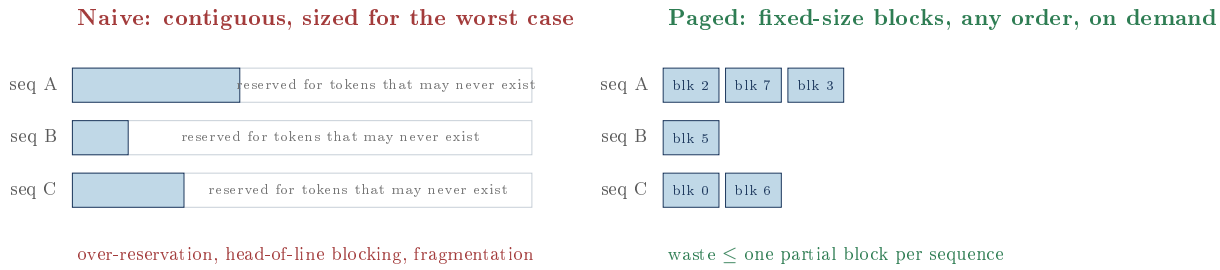

Figure 1: The reframing, borrowed from operating systems: a sequence stops owning a *region* and starts owning a *list of block numbers*.

Paging, and the property that matters most

Your operating system already does this. A program thinks it has one long stretch of memory; the OS hands it scattered fixed-size *pages* and keeps a *page table* translating what the program thinks it is using into where it really is. Here: a sequence thinks it has smooth token positions, the engine gives it scattered blocks (16 tokens each), and a *block table* translates.

The immediate wins are that external fragmentation becomes impossible (every free block is interchangeable) and waste is capped at one partial block per sequence. But the *valuable* consequence is subtler: once addresses are indirect, two sequences can point at the same physical block. Sharing becomes a refcount bump. Prefix caching and multi-completion forks both fall out of that for free.

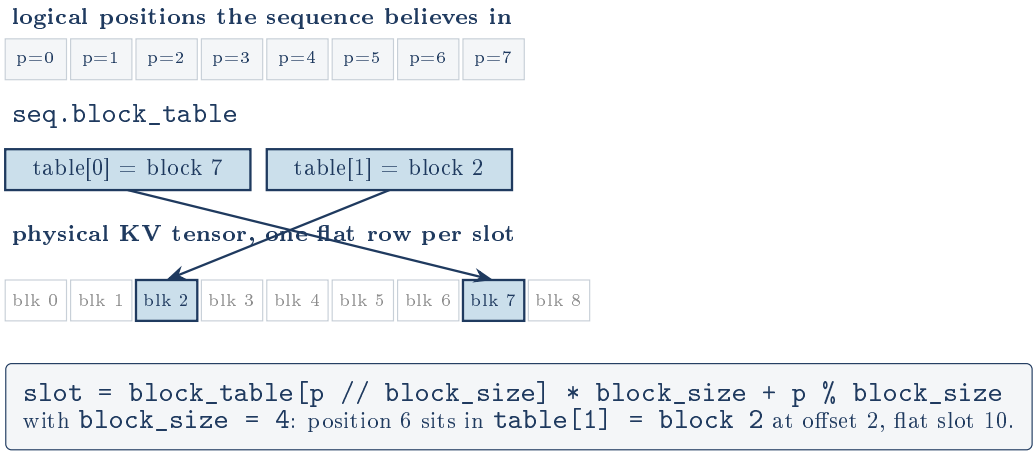

Figure 2: Address translation. One line of arithmetic, and everything else in the memory system is a consequence of it.

Three features are built directly on the indirection:

- **Copy-on-write.** Two sequences share blocks until one writes, then the writer takes a private copy of just that block. Requesting n completions of one prompt costs one block copy each, not n prompt recomputations. Full blocks stay shared forever: their contents can never change.
- **Prefix caching.** Full blocks are content-hashed with a *chained* SHA-256 (each block's hash includes the previous block's hash), so an identical token window under a different prefix cannot collide. A later request with the same prompt prefix re-references those blocks and skips computing those positions entirely. Freed hashed blocks go to an LRU evictable pool rather than the free list, so the cache costs nothing in capacity.
- **Cheap preemption.** A preempted sequence loses its blocks and recomputes later, and the prefix cache usually hands most of them back.

2.3 Idea two: batch continuously

Running several sequences through the model together is much faster than one at a time, because the weights are read from memory once and shared across the batch. The naive approach freezes a batch until every member finishes; since they finish at different times, most of the batch is computing padding for rows that are already done, and a new request cannot start at all.

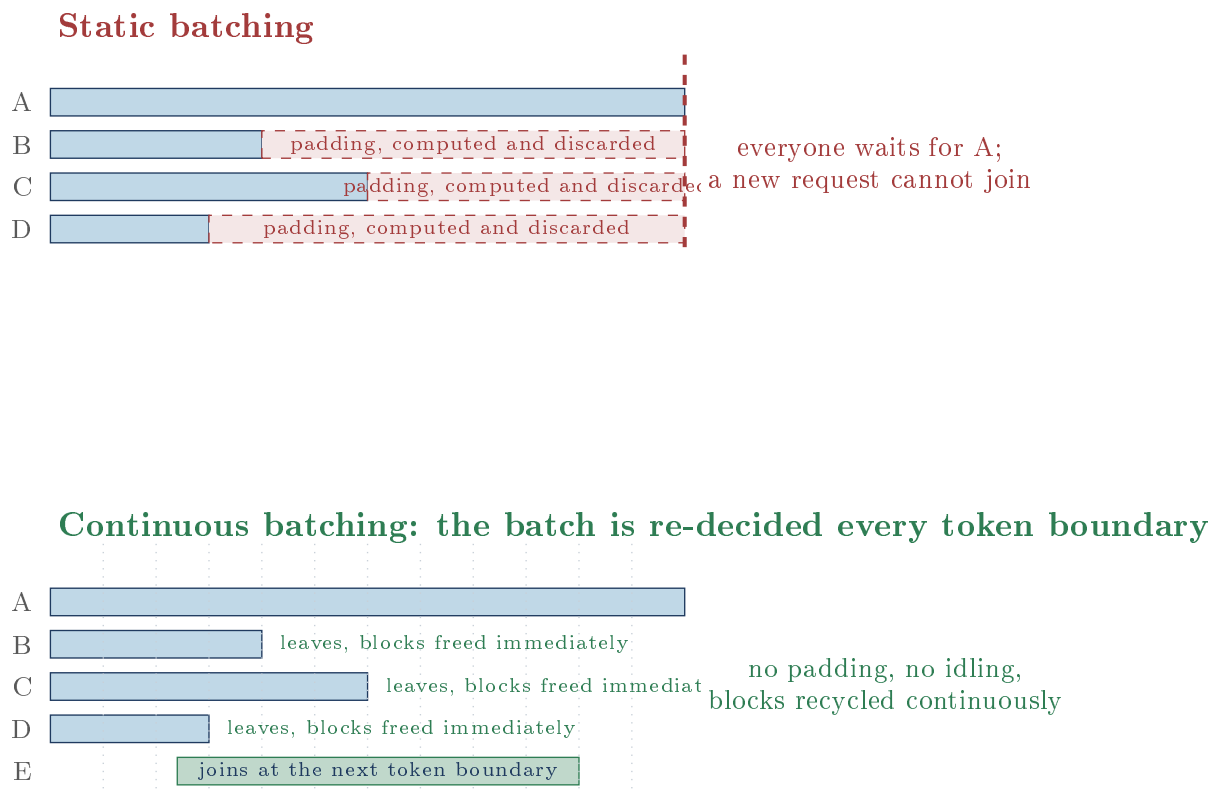

Figure 3: Where the project's measured 2.5x throughput gain comes from. Not from making any request faster: from never computing padding and never idling. Dotted lines are token boundaries, the only moments the scheduler may change its mind.

The scheduler (95 lines) admits FIFO while there is batch room and KV space, then enforces one hard invariant: **the entire decode batch must be able to grow by one token**.

Cumulative, not per-sequence: the bug worth knowing

The obvious check tests each sequence against the free list independently. With three sequences each needing one block and two blocks free, every individual check passes, the batch is admitted, and the third sequence crashes mid-step against an empty free list. The sequences are not competing with the free list; they are competing with *each other*, so the check must be summed across the whole batch.

When the summed check fails, the newest-arrived sequence is preempted: its blocks are freed and it is requeued *at the front* of the waiting queue. Sacrificing the newest protects the oldest, which has the most to lose; requeueing at the front stops it being starved by later arrivals. Recovery is by recomputation rather than swapping KV to slower memory, which on CPU would be a no-op with extra bookkeeping.

2.4 Idea three: speculate

Why guessing ahead can be free

Decoding is *memory-bound*. Producing one token means reading every weight in the model out of memory, and the arithmetic units idle through most of it. Running the model over five positions costs barely more than over one: the weights are read once either way.

So let a small cheap model guess the next k tokens, then run the big model *once* over all k guesses and ask "would I have said that?" for each. Every agreement is a token that cost a cheap forward instead of an expensive one.

The subtlety is that this must not change what the user gets. The accept/reject rule (Leviathan et al., 2023) guarantees it: accept a drafted token with probability $\min(1, p/q)$ where q is the draft's probability and p the target's; on the first rejection, sample a replacement from the residual $\text{normalize}(\max(p - q, 0))$ and discard the rest. If the target liked the token less than the draft did, keep it only p/q of the time, which is exactly the fraction that restores the target's rate; the residual is exactly the shape of the resulting deficit. The two paths sum back to p precisely.

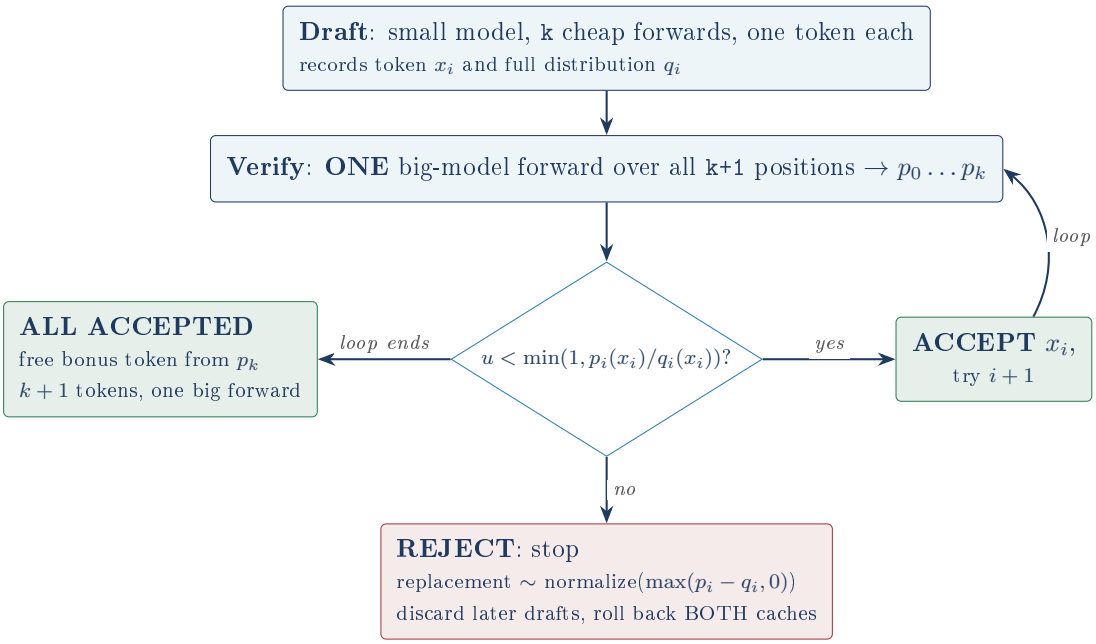

Figure 4: The accept/reject flow. The output is distributed *identically* to sampling from the target alone, not approximately. Speculative decoding trades no quality for speed. At temperature 0 both distributions are one-hot and the rule degenerates to "accept while the draft's argmax equals the target's".

3 Architecture

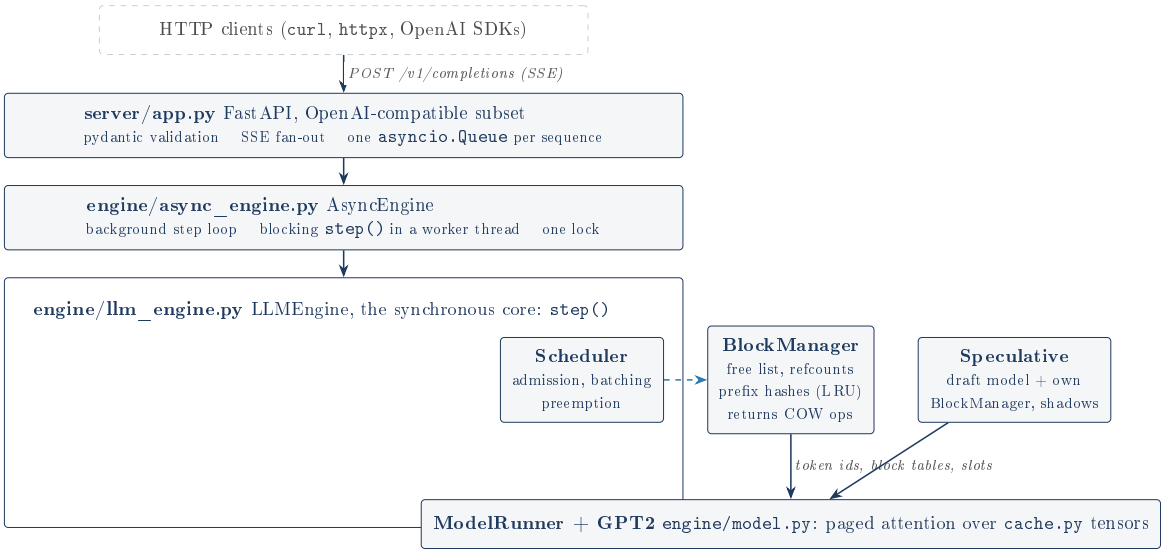

Figure 5: Four layers, requests down and tokens up. `app.py` knows HTTP and not blocks; `async_engine.py` knows threads and not attention; `llm_engine.py` knows tokens and never touches a socket.

The best structural decision in the codebase

`engine/block_manager.py` does not import `torch`. It is pure bookkeeping: it hands out integers and returns *instructions* of the form "copy physical block 4 to block 9". `engine/cache.py` is the only file that touches KV tensors, and it obeys.
That is why the hardest logic in a serving engine is testable in isolation: 15 tests covering fragmentation, refcounting, copy-on-write, LRU eviction and rollback run in well under a second because they never load a model. This is the lesson from the project most worth reusing elsewhere.

One `step()` advances every live sequence by one token boundary: schedule (admitting arrivals, preempting under pressure), run each prefill alone, run all decodes as one batched forward, then sample, detokenize and handle stopping. Above it, `AsyncEngine` pushes the blocking `step()` onto a worker thread via `asyncio.to_thread` so the event loop stays free to stream, with a single lock serializing engine mutations. The engine never learns what HTTP is; it puts `StepOutput` objects into queues and `app.py` turns those into SSE frames.

4 How correctness is evidenced

This is the strongest aspect of the project and its most transferable habit: the claims are demonstrated at four different levels rather than asserted.

Level	Evidence
The model is right	`test_parity`: this from-scratch GPT-2 forward versus Hugging Face's, largest disagreement across all 50,257 logits under `1e-3` (README reports about `3e-5` on distilgpt2, `2e-4` on gpt2: float32 noise).
The cache is right	`test_paged_forward_matches_cache_free`: the paged path and the plain causal path agree to `1e-4`.
The allocator is right	`test_scheduler_invariants_under_churn`: 300 scheduling steps over six sequences in a deliberately undersized cache, asserting after *every* step that `used + free == total`. A hand-rolled property test.
The speculation is right	`test_empirical_distribution_matches_target`: two tiny random transformers, 13-token vocabulary, 4000 full speculative rounds, asserting total variation distance from the target's true distribution under 0.05 (the noise floor at that sample count is about 0.02, and the test says so). Repeated with temperature and top-k applied.

Verify against a reference before building on top

Establishing logit parity first means every later bug is a *systems* bug. When the paged attention path misbehaved, the question was never "did I get the transformer wrong?". That is why the parity test is the first one worth pointing at, despite being the least interesting to read.
The speculative test is the sophisticated one. A probabilistic guarantee cannot be checked by comparing two outputs, so the project shrinks the problem until statistics become decisive: 13 tokens instead of 50,257, a two-layer random transformer instead of GPT-2, 4000 samples. The threshold was derived from the expected noise, not tuned until it passed.

5 Measurements

All numbers are CPU-only on an Intel i7-1185G7 (8 threads), PyTorch float32, greedy decoding. They are relative comparisons; they say nothing about absolute serving performance. The committed JSON in `benchmarks/results/` records the machine for each.

5.1 What the project reports, and what a re-run found

The committed numbers were re-measured on the same machine while this document was written. That was possible only because the JSON records the host, an i7-1185G7, and the model weights were cached locally. Conditions differed in three ways worth stating: the re-run used `torch 2.10.0+cu128` and `transformers 5.14.1` (the original's versions are not recorded), the host had a desktop session and background processes, and PyTorch actually used **4** threads, since the i7-1185G7 has 4 physical cores behind its 8 logical ones.

Claim the README makes	README	Re-run	Verdict
Throughput at concurrency 8	113.5 tok/s	99.2 tok/s	13% lower
Continuous batching, 1 to 8	2.54x	**3.01x**	reproduced, stronger
Engine at 8 vs HF sequential	2.84x	**3.11x**	reproduced, stronger
Speculative vs plain	0.72x	**1.14x**	**did not reproduce**
Speculative acceptance rate	0.5404	**0.5404**	exact, to 4 decimals

The headline claims hold, and the README is conservative

113.5 tok/s did not reproduce exactly: this run measured 99.2 tok/s at the same concurrency, on a contended host with unknown version drift. The figure is the right order but should be quoted as "roughly 100 to 115 tok/s on a laptop CPU depending on conditions", not as a constant.

The 2.8x reproduced and came out at 3.11x. The engine beat sequential Hugging Face by more than claimed, because contention hurts a one-at-a-time baseline more than a batched engine, which is precisely the effect continuous batching exists to produce. The 2.5x batching claim likewise measured 3.01x. Both headline claims are conservative on this machine, which is the right direction for a claim to be wrong in.

The speculative result reversed sign, and it is the real discrepancy

The README's most distinctive claim is a negative one: speculative decoding is "~0.7x slower here . . . not a CPU speedup". The re-run measured **1.14x faster**. Plain decode's wall time worsened 41% (5.73s to 8.11s) while speculative's *improved* 11% (7.96s to 7.11s).

This is not a correctness problem, and the evidence is unusually clean: 322 tokens drafted and 174 accepted in both runs, to the token, six weeks and two library versions apart. Greedy decoding is deterministic, so the algorithm made bit-for-bit identical decisions; only the clock disagreed. The engine's determinism claim is confirmed about as strongly as it could be.

A uniform slowdown preserves a ratio, and this one did not, so the effect is differential. The coherent explanation is per-forward overhead: plain decode does many tiny forwards (one token each), speculative does far fewer, fatter ones (k draft forwards, then one target forward over k+1=5 positions). Anything raising fixed per-forward cost, contention or a different torch build, penalizes the many-small strategy and barely touches the fewer-large one. The same mechanism explains why concurrency 1 fell to 0.74x of the original while concurrency 16 held at 1.00x: big batches amortize that overhead, a batch of one cannot.

Ironically the README's own reasoning predicts this. It says speculative decoding "pays off when the target forward dominates draft cost", then commits firmly to a single sample of a quantity it had itself identified as conditional. The honest position is narrower and stronger than either the README's or a naive "it is actually faster": on this hardware at k=4 it sits near break-even, and which side you land on depends on the software environment. Two measurements exist, 0.72x and 1.14x, and neither is a property of the algorithm. What is stable, and defensible without qualification, is the acceptance rate and the exact output equivalence.

The harness does not record the versions its numbers depend on

`machine_info()` captures CPU model, cores, platform, Python version and `"device": "cpu"`. It records neither the `torch` version, the `transformers` version, nor `torch.get_num_threads()`: precisely the variables a throughput number depends on. The committed JSON therefore identifies the *machine* but does not permit reproducing the *measurement*, which is the point of committing it. It is why the discrepancy above cannot be fully attributed: the experiment that would settle it is not recoverable from what was recorded. Three more lines in `machine_info()` would have prevented that permanently.
(The committed JSON was restored byte-for-byte after the re-run; the repository's recorded measurements are untouched.)

6 Honest assessment

6.1 What is genuinely strong

The correctness evidence is real and layered rather than asserted. The `BlockManager`-holds-no-tensors split makes the hardest logic testable in under a second. Every figure in the README transcribes faithfully from the committed JSON, with no rounding in the flattering direction. And the project reports its own headline feature going backwards on CPU, with the reason, rather than quietly dropping it: a README that says 0.72x is more persuasive than one claiming 2x.

The pattern behind every serious finding below

All of them live in the space *around* the algorithms rather than in them. The paged cache, the scheduler, the accept/reject sampler: correct, well tested, defensible in detail. The failures are in dependency declaration, process lifetime, error propagation and cross-component invariants, exactly the set of things unit tests do not reach and a short benchmark cannot reveal.
That is a useful thing to be able to say about one's own work: the hard part was done well and the gaps are in the boring parts. It is also the honest answer to "what next", ahead of the GPU kernel.

6.2 The findings that matter

The top three were found by *running* the project, not reading it. The code reads well enough that none of them are visible from the source alone.

Any exception in `step()` permanently bricks the server (verified)

`AsyncEngine._run_loop` has no exception handling at all. It runs as a bare `asyncio.Task`; nothing awaits it, attaches a done-callback, or inspects its exception. So when `step()` raises, the loop dies silently and the server goes on accepting requests it will never answer, while `/healthz` keeps returning `ok`.
Driven end-to-end through `AsyncEngine`, the stream hung for 25 seconds with no tokens, the task was `done()` with `RuntimeError('draft KV cache exhausted')`, and then a subsequent, entirely innocent `"Hello there"` request *also* hung: one bad request bricks the process until someone restarts it. This ranks first because it is a severity multiplier, converting every other bug (including ones not yet found) from a bad response into a total outage. The fix is routine: wrap the step, fail the affected sequences, keep looping.

The draft cache's capacity invariant is false and reachable (verified)

`docs/ARCHITECTURE.md` states that the draft cache's "block usage is always a subset of the target's; one capacity check then covers both caches", and a matching comment sits in `llm_engine.py`. The scheduler indeed checks only the target's block manager.

The reasoning covers *retention* (the draft returns freed blocks immediately rather than parking them in an evictable pool), and that much is true. It omits *sharing*, which runs the other way: the target's `fork()` path lets sequences share physical blocks, while the draft allocates private blocks for every shadow. The draft therefore needs *more* blocks than the target, the opposite of the claimed subset relation. Note this does not even depend on prefix caching, so the stated *reason* ("runs without prefix caching") does not deliver the invariant on its own terms.

Driven with `num_blocks=12`, a 63-token shared prompt and `n=4`: step one showed four sequences sharing four blocks (the sharing working exactly as designed), the target-side check passed comfortably, and step two died with `RuntimeError: draft KV cache exhausted` because the draft needed sixteen blocks against twelve. Combined with the finding above, that is a remotely triggerable permanent outage.

At the default configuration arithmetic saves it by luck, not by the invariant: `max_batch_size` 8 times `max_model_len` 1024 is exactly the 8192 token slots the cache holds. Raise the batch size and the margin is gone; the project's own benchmark harness starts its server with `--max-batch-size 16` (though never with a draft model). This is the most serious *design* finding because it is a documented invariant that the code does not have: answering "why is one check enough?" from the architecture document would be giving a wrong answer.

`requirements.txt` declares a range the project cannot run in (verified)

`engine/model.py` calls `from_pretrained(model_name, dtype=...)`. The `dtype` keyword is recent; Hugging Face spelled it `torch_dtype` for years. `requirements.txt` declares `transformers>=4.44,<6`. Bisected against the real package index: 4.51.3 and 4.55.4 both fail with `TypeError: GPT2LMHeadModel.__init__() got an unexpected keyword argument 'dtype'` and load no model at all, while 4.56.0 and 5.14.1 work. The true floor is **4.56.0**.

So every version from 4.44 to 4.55.4, roughly two years of releases and the larger part of the declared range, yields a clone that cannot load a single model. This surfaced on the machine used for this document, which had 4.51.3 installed system-wide: the entire model-backed suite errored out. The fix is `transformers>=4.56,<6`. The finding is not the severity but the category: the project's central claim is that nothing is asserted that was not verified by running it, and its declared environment is not the environment it was verified in.

A verified memory leak in the long-running part

`LLMEngine.add_request` inserts a `_SeqState` per sequence. `abort()` removes it for sequences still waiting; `_finish()`, the path every normally completing sequence takes, never does. So a server retains one entry per request served, forever, each holding that request's complete generated text. Confirmed by driving it, not by reading: after twenty sequential generations the scheduler's queues were empty and the block manager was back to a full free list, while `seq_states` still held twenty entries. Nothing catches it because every test and benchmark constructs an engine, runs a few requests, and exits: precisely the blind spot of a project about long-running services.

Seeded determinism has a load-dependent hole for `n > 1`

When a sibling completion cannot join the batch, the code has *already* consumed a random draw from its generator sampling a token it then discards before requeueing. The same request with the same seed therefore produces different output depending on whether the batch happened to be full. Seeded determinism is an advertised feature; the tests use a batch size that never triggers the branch, and the multi-completion HTTP test checks that two choices come back but never that they reproduce.

The attention is not paged at the kernel level

The README discloses this and the code confirms it exactly: `gather` pulls each sequence's scattered KV into a *dense padded* tensor and calls a standard attention kernel. The **memory** savings are entirely real; the **compute** savings are not, since a batch mixing a 500-token sequence with seven 20-token ones pads all eight to 500. The gather also materializes a copy of the whole batch's context every layer, every step. A real engine writes a fused kernel that walks the block table inside attention. The stated reason for not doing so (on CPU with a small model it was not the bottleneck) is defensible, and this is the biggest gap between the project and the systems it studies, so it is worth volunteering rather than being caught by.

The TTFT curve is a readout of unbatched prefill

Time to first token goes 0.069s, 0.266s, 0.541s, 1.266s at concurrency 1, 4, 8, 16: near-linear. Prefills run one at a time, and a single prefill of these prompts takes about 0.07s, so eight simultaneous arrivals means eight sequential prefills and the last client waits for all of them ($8 \times 0.07 \approx 0.55$s against a measured 0.541s). The README calls the growth "the expected tradeoff" and separately lists unbatched prefill under future work, without connecting the two. The curve is not a property of continuous batching; batching ragged prompts would flatten most of it. *(This arithmetic is inferred from the committed JSON and the code, not separately measured.)*

Smaller findings, in descending order

The incremental detokenizer decodes the *entire* output every step and slices off what was already sent, making it $O(n^2)$ in output length (invisible at the benchmark's 32 tokens, half a million token-decodes at GPT-2's 1024 limit). `_match_prefix` recomputes its whole SHA-256 chain three times per admission. `EOS_TOKEN_ID = 50256` is hardcoded at module level rather than read from the tokenizer that the engine already holds, making the engine silently GPT-2-family-only in a way the configuration does not express. `num_speculative_tokens` is applied in `add_request` even with no draft model configured, while `Scheduler.lookahead` correctly is not: two places disagreeing about the same question. `k` is a minimum across the batch, so one nearly-finished sequence throttles everyone's lookahead. A just-admitted prompt can be preempted in the same `schedule()` call that admitted it (correct, but wasted work). And the prefix cache trusts SHA-256 without ever comparing tokens on a hit: the right call, but its correctness rests on a cryptographic assumption rather than a check.

6.3 What the benchmarks do and do not support

"2.8x over baseline" is honest, conservative, and not controlled

It compares 113.5 tok/s (this engine, 8 concurrent, driven over HTTP with SSE parsing) against 40.0 tok/s (HF `generate()`, in-process, no serialization). Both are real and both are in the committed JSON, but they come from different harnesses. The bias favours the baseline, so the true gap is if anything wider, which is the right way to be wrong. The honest framing is "2.8x end-to-end including our serving overhead against a bare in-process loop". Note too that sequential HF `generate()` is a weak baseline by construction; the strong one, HF with its own batching, is not measured.
Two further caveats: the sweep's harness counts *SSE events*, not tokens (one event is one token for plain decoding, so the totals are right, but the same harness run with a draft model would understate throughput by up to 5x with no warning); and the speculative benchmark measures a warm prefix cache, since it warms up on the same four prompts it then times. The latter is fair between the two engines it compares, which is what that benchmark is for.

The gap the project cannot close, and says so

There is no comparison against a production engine like vLLM, because vLLM needs a GPU and this machine has none. The README states this as absent, with the reason, rather than fabricating a number: the right call, since a made-up figure would be worse than a missing one. But it means the project can demonstrate that its *mechanisms* work and cannot demonstrate that they work *well*. "Continuous batching gives 2.5x on 8 CPU threads" is what was measured; nothing here says how the design behaves where these designs actually live.

6.4 The design choices worth defending as made

- **One coarse engine lock.** Correct here: the forward dominates step time so completely on CPU that contention is unmeasurable, and one lock is the version a person can reason about. It stops being right the moment you want to overlap prefill with decode or pipeline the sampler.
- **Recompute-only preemption.** On CPU the "device" and host are the same memory, so swapping KV out would be a no-op with extra bookkeeping.
- **Unbatched prefill.** Defensible on CPU (prefill is already compute-bound), and the honest cost is the TTFT curve above.
- **Speculative decoding kept despite going backwards.** It is correctness-verified and 0.72x on CPU, because the draft's forward is not cheap relative to the target's. It pays off when the target dominates draft cost (large model, GPU). Measuring it honestly and finding it negative is more useful than assuming the paper's GPU numbers transfer, and that is the right thing to say about it.